A Unified Software Architecture Theory

Tomer Ben-David, AI

April 2026

Abstract

Modern software systems often become harder to verify, operate, and evolve not simply because scale increases, and not because individual technologies are weak, but because core business invariants are spread across too many independent infrastructure boundaries. When one business operation is implemented across multiple services, queues, and orchestration layers, it becomes harder to keep business logic, authoritative state, and transition rules coherent.

This whitepaper presents the Unified Software Architecture Model (USAT), a practical architectural model for organizing business authority in modern software systems that distribute one business operation across many technical boundaries. USAT improves correctness, operational legibility, developer experience, and long term maintainability by keeping business logic inside cohesive application services, by placing authoritative truth in a directly queryable system of record, and by assigning asynchronous and analytical systems to clear supporting roles. The central claim is simple: software architecture should be organized around business intent and invariants first, and around infrastructure primitives second.

USAT is demonstrated through a high stakes Autonomous Financial Risk & Payment Orchestration (AFRP) example that combines strict financial consistency, asynchronous document and fraud workflows, human review steps, and real time dashboards. The whitepaper argues for a clear separation between a transactional core, an event driven action layer, a real time analytical read path, and a batch analytical layer.

The goal is not to reject distributed systems or cloud native tooling. The goal is to place them behind a clear architecture of authority, responsibility, and data flow so that the system remains understandable, queryable, and correct as it grows.

Executive Summary

This whitepaper presents an intent first architecture model for modern software systems, especially systems that distribute work across queues, background jobs, workflow tools, callbacks, and analytical paths in ways that fragment business logic, authoritative state, and operational understanding. It is particularly useful for systems that must preserve strict business invariants while also supporting asynchronous processing and large scale analytics, but the underlying design rule is broader: business invariants should determine where authoritative truth and transactional authority live, while asynchronous actions, dashboards, and historical analysis should scale outward from that foundation rather than fragment it.

USAT is aimed at teams building systems where correctness, troubleshootability, operator legibility, and long term maintainability matter as much as throughput and scale. Its value is not in introducing a new infrastructure primitive, but in providing a disciplined way to place boundaries so the system stays understandable as it grows.

Summary

- Start from business invariants and define what the system must keep true.
- Keep core business logic inside cohesive application services rather than spreading it across queues, workflow tools, and infrastructure callbacks.
- Decide where authoritative business truth lives and keep that truth directly queryable.
- Use a transactional bridge to bind committed state and outbound intent without letting transport systems own the business process.
- Let asynchronous workers and external inputs react to committed truth and re enter the core only through explicit local commits.
- Separate transactional truth, action systems, dashboard systems, and historical analysis into distinct operational planes.

- Optimize for correctness, legibility, troubleshooting, and long term maintainability together.

1. Introduction

The core problem this whitepaper addresses is not infrastructure choice by itself. It is the loss of coherent business truth when one business operation is distributed across services, queues, workflows, callbacks, and analytical systems without one clear authority for state, rules, and transitions.

USAT does not argue that all processing, teams, or systems should collapse into one place. It argues that even in a distributed architecture, each important business operation still needs one clear authority for the state, rules, and transitions that must remain coherent.

In many real systems, especially cloud native ones, those invariants become fractured across network boundaries. A workflow that is conceptually one business action is implemented as a chain of API calls, message publications, background workers, third party callbacks, and analytical projections. Each component may work correctly in isolation while the overall business operation becomes difficult to reason about, difficult to debug, and difficult to keep consistent.

This whitepaper argues for an intent first architectural model.

1. Core business operations should be identified explicitly.
2. Business logic should remain in cohesive application services rather than being spread across infrastructure choreography.
3. Operations that must succeed or fail together should be committed locally within one transactional boundary whenever practical.
4. Asynchronous side effects should be derived from committed state rather than interleaved with the commit through unreliable network hops.
5. Operational truth should remain directly explainable from the system of record rather than reconstructed from scattered infrastructure history.
6. Read optimized and action oriented downstream systems should be separated from the transactional write path.

This is not an argument for a return to an undifferentiated monolith, nor is it an argument against distributed systems. It is an argument that the placement of boundaries should be determined by invariants and authority, not by fashion or default infrastructure decomposition.

The whitepaper proceeds in four steps. It first defines the failure mode of fragmented business truth, then identifies the components of the model, then explains how business logic ownership, operational legibility, and transactional authority work together at the core, and finally shows how transactional bridging, asynchronous actions, and analytical systems scale around that core without obscuring it.

2. Scope and Central Claim

The Unified Software Architecture Model (USAT) is a practical design model for modern software systems whose business operations cross several technical boundaries and whose teams still need one coherent way to place authority, explain current state, and evolve the system safely.

It is especially useful when a system combines:

- business flows whose state, rules, and transitions must remain understandable and authoritative,
- asynchronous, operational, or analytical subsystems that react to those flows,
- and a need to keep business truth legible even as the surrounding architecture becomes more distributed.

USAT is most relevant for architects and senior engineers working on systems where several concerns must coexist:

- strong business invariants,
- asynchronous workflows,
- operator facing legibility,
- real time dashboards,
- and long term maintainability under continued product growth.

Typical examples include payments, risk and compliance platforms, order and inventory systems, logistics workflows, and other software where business truth must remain queryable even as the surrounding system becomes highly distributed.

The central claim of USAT is:

> When a business operation contains a small set of state transitions that must remain mutually consistent, that operation should have one authoritative transactional center. Distribution should happen around that center, not through it.

That center is not only a storage boundary. It is also the place where the owning service keeps the business rules, where authoritative truth remains queryable, and where operators can explain what the system currently believes.

USAT therefore prefers:

- explicit identification of business invariants and transactional authority,
- cohesive service owned business logic rather than infrastructure owned business flow,
- a transactional core for business truth,
- a transactional bridge from committed state to asynchronous side effects,
- separate downstream systems for action processing and analytics,
- explicit boundaries between business state, public event contracts, and analytical projections,
- and a system shape that remains operationally legible to engineers and operators.

The contribution claimed here is modest and practical. This whitepaper does not claim a new formal theory of distributed systems. It proposes a unifying architectural model that combines several known ideas into one decision framework centered on business invariants.

2.1 Components of USAT

USAT is made up of eight connected components:

1. **Business invariants** These define what must remain true for the business to remain correct.
2. **Service owned business logic** Business rules and state transitions should live in cohesive application services rather than being scattered across queues, workflow definitions, or transport tools.
3. **Transactional center** This is the narrow architectural boundary where mutually dependent state transitions are committed together.
4. **Transactional bridge** This is the mechanism that binds committed business truth to outbound intent, typically through the transactional outbox or an equivalent local commit pattern.
5. **External re entry rule** Asynchronous inputs such as callbacks, manual review decisions, or processing results become authoritative only when they are accepted through a new local commit.
6. **Operational legibility** Engineers and operators should be able to explain current business state and pending actions from the authoritative system of record.
7. **Four plane model** Transactional truth, asynchronous actions, real time analytical reads, and long horizon analytics are separated into distinct operational planes.
8. **Contract separation** Public event contracts are separated from internal domain table replication so internal schema evolution does not leak into external interfaces.

3. Running Example of Autonomous Financial Risk and Payment Orchestration

To make the model concrete, consider an Autonomous Financial Risk & Payment Orchestration (AFRP) system.

This system must support all of the following:

- users uploading invoices or proof of funds documents,
- asynchronous OCR and document extraction,
- fraud and risk analysis,
- manual compliance review for flagged cases,
- payment authorization and settlement tracking,
- asynchronous bank callbacks or settlement imports,
- executive and customer dashboards with low latency reads.

This is a useful stress case because it combines several forces that often compete:

- financial correctness,
- asynchronous external dependencies,
- human review loop state transitions,
- high concurrency read traffic,
- long term analytical and audit requirements,
- ease of troubleshooting across asynchronous boundaries,
- architectural clarity for developers and operators,
- and long term maintainability as the system evolves.

The challenge is not only moving data through the pipeline. The challenge is preserving a coherent business truth while the system interacts with many asynchronous actors.

To make the example concrete, the whitepaper assumes a small set of explicit invariants:

1. A cleared risk decision must never exist without an authoritative payment record or an explicit recorded reason why payment did not proceed.
2. A settlement update must never silently overwrite an incompatible prior payment state.
3. A manual review decision must become queryable business truth in the core system rather than remain trapped inside a workflow tool.
4. A support or operations engineer should be able to explain the current state of a document case from the authoritative system of record without reconstructing the flow from multiple infrastructure logs.
5. The main business decisions for the case should remain in cohesive service logic rather than being hidden inside queue topology, workflow definitions, or infrastructure callbacks.

For the rest of this whitepaper, the AFRP system serves as the running example used to anchor the model concretely.

4. Positioning Relative to Existing Architectural Ideas

USAT is best understood as a synthesis and prioritization of existing architectural ideas rather than a rejection of them.

It is closely aligned with:

- modular monolith thinking for strongly coupled transactional flows,
- transactional outbox patterns for reliable state to event bridging,
- CQRS separation between write models and read models,
- CDC based replication for analytical and search oriented downstream systems,
- stream processing for asynchronous reaction and policy enforcement,
- data warehouse and lakehouse patterns for historical analysis.

USAT differs mainly in emphasis. It treats these not as independent tools to be adopted opportunistically, but as parts of one ordered model:

1. Establish the transactional center around business invariants.
2. Keep the important business rules in cohesive services and keep the resulting truth directly explainable.
3. Commit business truth locally.
4. Export intent and state outward deliberately.
5. Let specialized downstream systems scale around that center.

Representative sources for those patterns are listed in the References section and are cited where the whitepaper makes direct use of those ideas. In a more formal publication, this positioning section should also include a deeper comparative discussion of where USAT agrees with, narrows, or departs from those prior approaches.

5. The Problem of Fragmented Business Truth

Many systems are built by mapping each workflow step directly onto an infrastructure primitive. A request writes a database row, publishes to a queue, triggers object storage events, starts a workflow engine, calls third party APIs, and updates dashboards through multiple downstream systems.

That style can scale operationally, but it has a common failure mode: the business process becomes distributed in ways that separate invariants, authority, and transition logic from the services that should own them, and the business logic itself starts to migrate out of cohesive services and into infrastructure choreography.

5.1 A Common Failure Pattern

In a typical fragmented implementation:

1. The API writes a `PENDING` row.
2. The API tries to publish an event to a queue.
3. The user uploads a file to object storage, which emits a storage event.
4. A worker starts an OCR or fraud workflow.
5. Another worker calls an external risk provider.
6. Another service creates a payment intent.
7. A later callback updates the payment state.
8. A manual reviewer may approve or reject the case through a different tool.

Each step is individually reasonable. The problem is that the system may no longer have one place that can answer basic operator questions with confidence:

- What is the current business state?
- Why is it in that state?
- Which next actions are pending?
- Which actions were already committed?
- Which transitions are valid from here?
- Where does the actual business logic for this transition live?

When those answers are spread across queue state, workflow history, object storage notifications, service logs, and partial database rows, the system becomes harder to operate than the business problem itself.

5.2 The Dual Write Failure

The most familiar instance of this problem is the dual write problem (see References 4, 6, 7, and 8):

- the application commits business state in the database, and
- the application independently publishes the corresponding message over the network.

If one succeeds and the other fails, the system diverges. Retries, tracing, and orchestration can reduce the blast radius, but they do not remove the underlying inconsistency window.

5.3 What This Whitepaper Calls Implanted Architecture

This whitepaper uses the term **Implanted Architecture** for an architecture in which core business intent is embedded primarily in infrastructure choreography rather than in a cohesive, queryable application state model.

The term is not meant as a blanket rejection of queues, workflow engines, or cloud primitives. It names a specific failure mode: infrastructure becomes the place where the business process actually lives.

The first figure is intentionally simple. It shows how the workflow fragments structurally. The deeper problem is not only component count. It is that authority and explanation become scattered across several systems. The second figure makes that failure mode explicit.

6. The USAT Response

USAT starts from a simple question:

> Which state transitions must remain mutually consistent for the business to remain correct?

Those transitions define the **transactional core**. The related business logic should remain in the services that own those transitions rather than being dissolved into transport systems.

That response has three parts:

- keep the important business reasoning in cohesive services,
- keep authoritative truth and auditability in one queryable transactional boundary,
- let asynchronous and analytical systems scale around that boundary in explicitly supporting roles.

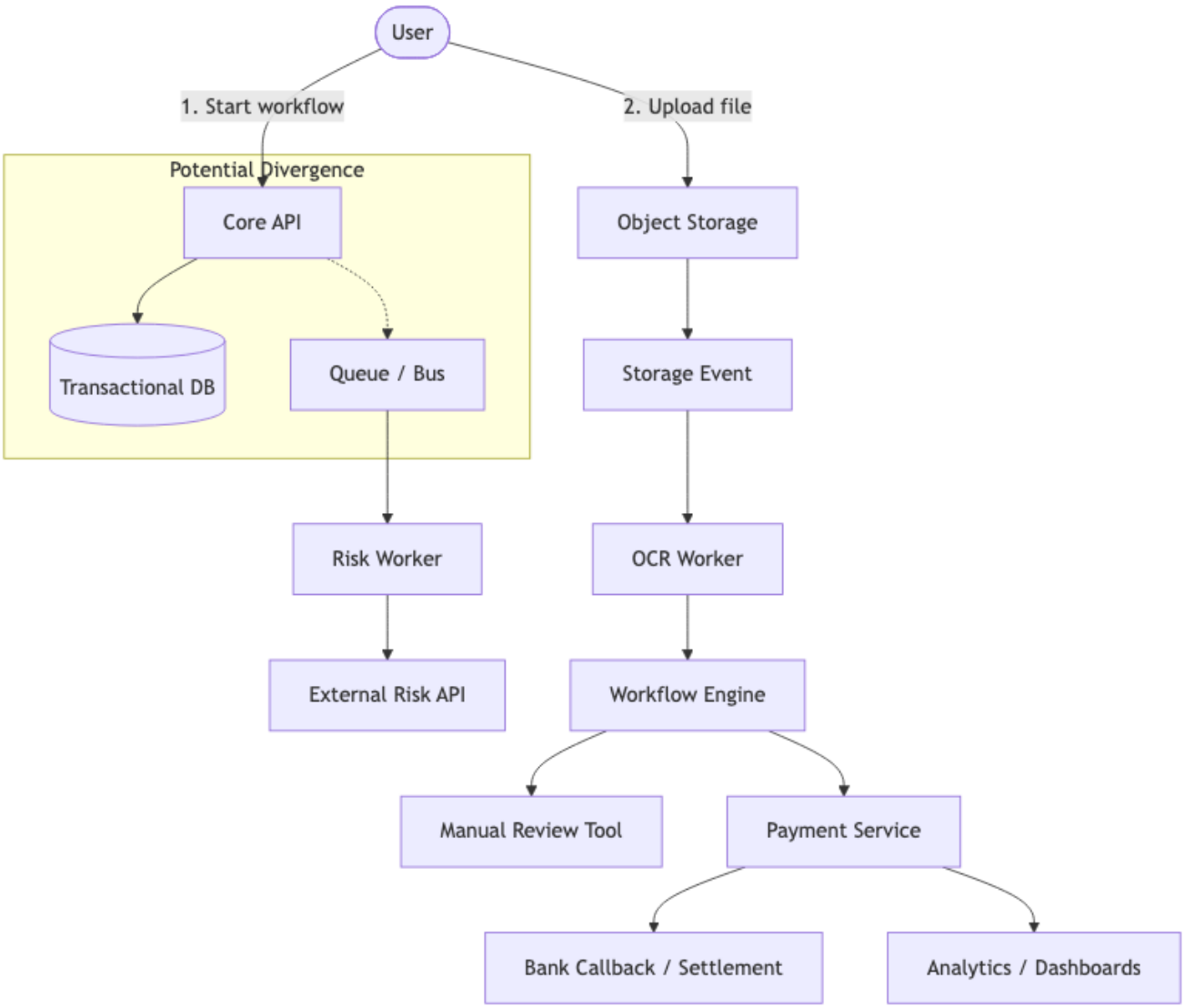

Figure 1: Fragmented workflow diagram

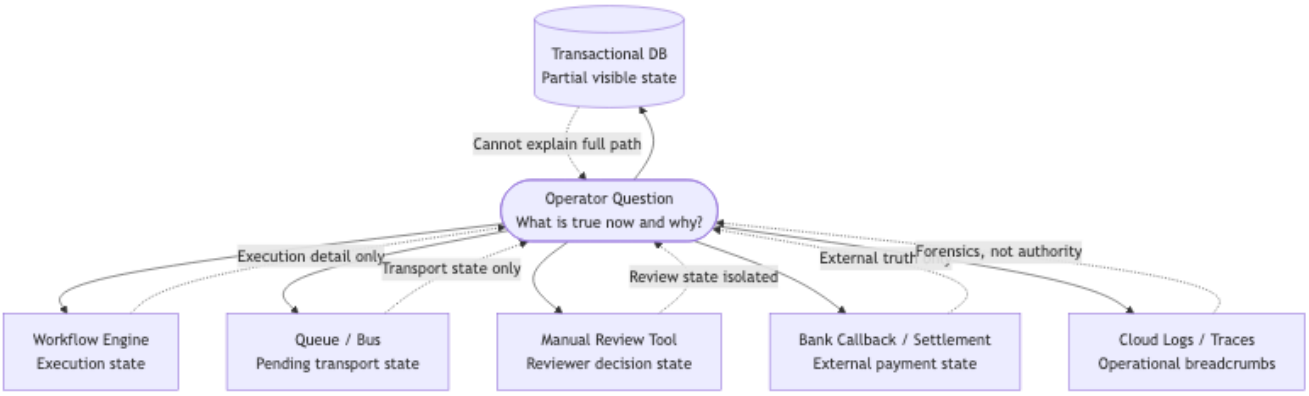

Figure 2: Authority and blind spots diagram

In the AFRP example, some operations are naturally asynchronous, such as OCR, external fraud scoring, or bank settlement callbacks. But when those asynchronous inputs are accepted as new business truth, that acceptance should happen through one local database transaction that records:

- the authoritative state mutation,
- the audit context,
- the publish intent for downstream consumers.

This gives the system one queryable center of truth while still allowing the rest of the architecture to scale outward. It also preserves a clear place where the business logic can be read, tested, and reasoned about as application code rather than inferred from infrastructure behavior.

In AFRP terms, the transactional center would typically include entities such as the document case, the risk decision state, the payment intent or ledger mutation, and the audit trail required to justify those transitions.

7. Service Owned Business Logic and Operational Legibility

USAT becomes much weaker if it is reduced to a commit pattern. The model is also about where business reasoning lives and how the system remains explainable after it becomes distributed.

7.1 Service Owned Business Logic

The core business rules of a workflow should live in cohesive application services that own the relevant state transitions.

That means the service code should answer questions such as:

- when a case is allowed to move from `UNDER_REVIEW` to `RISK_ACCEPTED`,
- when a payment authorization may be created,
- when a settlement callback is compatible with the current state,
- when manual review is required before any further action is taken.

Those rules should not be hidden primarily inside:

- queue topology,
- workflow engine definitions,
- storage event chains,
- retry configuration,
- or infrastructure specific callback glue.

Infrastructure can trigger work, transport events, and coordinate background execution. It should not become the place where the business policy itself is encoded.

In AFRP, the fact that a bank callback arrived is not itself the business decision. The business decision is whether that callback is valid for the current payment state, whether it advances the case, whether it requires review, and what new authoritative truth should be recorded. That reasoning belongs in the service that owns payment and case state.

A useful design check is simple: if the team has to inspect queue graphs, workflow history, and cloud event chains to explain why a case entered its current state, then business logic has already leaked too far into infrastructure.

7.2 Operational Legibility

USAT also insists that a production system remain legible to the people operating it.

Operational legibility means a support engineer, operator, or developer should be able to answer the following from the authoritative system of record and its directly attached audit trail:

- what the current business state is,
- why the system believes that state is correct,
- which external or asynchronous inputs have already been accepted,
- which next actions are pending or already requested,
- and which service owned rule produced the last important transition.

This requirement is not cosmetic. It changes architecture.

If the only way to explain a case is to combine database rows, queue offsets, workflow engine history, object storage notifications, and ad hoc service logs, then the architecture is no longer preserving a coherent operational model.

For AFRP, operational legibility means an operator should be able to inspect a document case and see, in one authoritative place, the accepted risk decision, the payment state, the last reviewed action, the audit justification for the last transition, and the downstream intent that was already committed. The operator may still use workflow tools and dashboards for context, but those tools should enrich the picture rather than define the truth.

Taken together, service owned business logic and operational legibility explain why USAT is broader than a durability pattern. The model is trying to preserve coherent business reasoning, clear authority, and explainable state as the system grows.

8. The Transactional Bridge

The main mechanical pattern in USAT is the transactional outbox.

8.1 Why the Outbox Matters

The outbox is not primarily about elegance. It is about binding two things together in one commit:

- "the business state changed", and
- "the system intends to publish that fact."

Instead of updating the database and separately publishing to Kafka, SQS, or another bus, the application writes the business change and the outbound event record in the same local transaction. A publisher or CDC process then relays the outbox record downstream, following the same general pattern described in transactional outbox guidance and Debezium's outbox tooling (see References 4, 6, 7, and 8).

If the transaction fails, neither state nor intent is committed. If the transaction succeeds, both are durable.

8.2 Canonical Local Commit

```sql
BEGIN;

UPDATE payments
SET status = 'AUTHORIZED'
WHERE id = 'pay_789';

INSERT INTO payment_audit_log (payment_id, actor_id, action, timestamp)
VALUES ('pay_789', 'user_42', 'PAYMENT_AUTHORIZED', now());

INSERT INTO payment_outbox (aggregate_type, aggregate_id, event_type, payload, metadata, created_at)
VALUES ('Payment', 'pay_789', 'PaymentAuthorized', '{...}', '{...}', now());

COMMIT;
```

This pattern gives three useful properties:

1. The business state is immediately queryable in the transactional store.
2. The audit trail is recorded at the same boundary as the state mutation.
3. The intent to notify downstream systems is durable and can be delivered asynchronously.

8.3 USAT Is Broader Than Outbox

The transactional outbox is an important mechanism in this whitepaper, but it is not the whole contribution.

USAT is broader in five ways:

1. It starts with **business invariants**, not with messaging patterns.
2. It keeps **business logic in cohesive services**, not in queues, workflow definitions, or infrastructure callbacks.
3. It defines a **transactional center** where authoritative business truth becomes queryable.

4. It preserves **operational legibility**, so engineers and operators can explain current state without reconstructing the full history from scattered infrastructure tools.
5. It separates the system into distinct **operational planes**, so action systems, dashboard systems, and historical analysis systems do not blur together.

In other words, the outbox is the bridge. USAT is the architecture around that bridge.

8.4 Terminology Clarification

The outbox entry is written **within** the same local transaction as the business mutation. The relay to external infrastructure happens **after** commit. For that reason, the useful distinction is:

- **outbox write inside the transaction**, followed by
- **publication after commit**.

Calling that row "after commit" would be imprecise and should be avoided.

8.5 Public Events and Domain CDC

A common alternative is to stream changes directly from domain tables through CDC. That can be useful, but it should not automatically become the public event contract. That distinction follows naturally from outbox and CDC patterns: one exists to publish intentional external events, while the other is often used for replication and projection (see References 4, 6, 7, and 8).

If downstream consumers depend directly on internal table schemas, then ordinary refactoring of internal columns becomes externally breaking behavior.

USAT therefore separates:

- **Outbox events for public or boundary crossing intent**
- **Domain table CDC for internal replication and analytical synchronization**

This distinction keeps internal schemas free to evolve while maintaining stable external contracts.

8.6 When Direct Domain CDC Is Appropriate

Direct domain CDC remains useful for internal use cases such as:

- replicating dimensions into analytical stores,
- hydrating search indexes or read replicas,
- forensic row level auditing.

The rule is simple:

- use the **outbox** for intent you want other systems to consume as a stable contract,
- use **domain CDC** for internal state replication.

For AFRP specifically, the outbox would carry events such as `RiskReviewRequested`, `RiskDecisionRecorded`, `PaymentAuthorized`, or `SettlementRecorded`, while domain CDC might still replicate normalized tables like `customers`, `vendors`, or `payments` into analytical systems for joins.

9. External Inputs

Not every signal originates inside the transactional core. Some signals arrive from outside it:

- object storage notifications,
- bank webhooks,
- settlement files,
- manual reviewer actions.

Those signals are not in the same database transaction as the core system, and USAT does not pretend otherwise.

The rule is instead:

> When an external or asynchronous signal changes business truth, the receiving boundary must perform one local transaction that records the new truth, the relevant audit context, and any resulting publish intent.

That applies to:

- OCR completion becoming accepted structured data,
- manual approval or rejection,
- bank settlement or failure notifications.

This includes handling asynchronous issues gracefully. If a downstream worker fails, that failure does not roll back an already committed core transaction. Instead, the failure must be handled through an explicit follow up path such as retry, manual review, or a new domain level state transition recorded through another local commit. The key point is not that every failure becomes the same business state. The key point is that once a failure becomes authoritative business truth, it should be recorded through the same clear transactional boundary.

This is a subtle but important point. USAT does not eliminate asynchrony. It gives asynchrony a clean point at which it becomes authoritative business state, without artificially conflating infrastructure failure with domain failure.

10. Shared Database and Distributed Sagas

USAT does not claim that all systems should share one database. It makes a narrower claim:

When two or more domain operations must remain strongly consistent within one user visible business action, physically splitting them across independent database commits often introduces avoidable complexity.

In those cases, a modular monolith or shared transactional core may be the simpler and safer design, especially when the alternative is to push one user visible operation across saga style coordination boundaries (see References 1, 4, and 5).

That means:

- logical module boundaries should remain strict,
- code ownership should remain explicit,
- but physical database isolation should not be treated as a virtue by default.

The tradeoff is real. Shared databases can constrain autonomous deployment and increase coupling if boundaries are weak. But distributed sagas also have a cost: more inconsistency windows, more compensation logic, more operational debugging, and more difficulty establishing what the business currently believes to be true. Those tradeoffs are well aligned with the general strengths and weaknesses discussed in modular monolith and saga literature (see References 1 and 5).

USAT prefers to pay the local modularity cost when the alternative would be to distribute an operation whose correctness depends on atomicity.

In AFRP, that means risk acceptance and payment creation may belong to one transactional center if the business cannot tolerate a cleared risk state without a corresponding authoritative payment record. If the business can tolerate eventual consistency there, the boundary could be split. The architecture should follow that business fact rather than a preset doctrine.

11. The Four Plane Model

Once the transactional center is defined, USAT separates the rest of the system into four operational planes. The clearest way to understand those planes is not as one more workflow, but as four distinct ownership boundaries that sit side by side around the same business truth.

The four plane model is as central to USAT as the transactional bridge. It defines where different kinds of responsibility should live:

- Plane 1 owns authoritative business truth.
- Plane 2 owns asynchronous reactions and automated actions.
- Plane 3 owns fast analytical reads for dashboards.

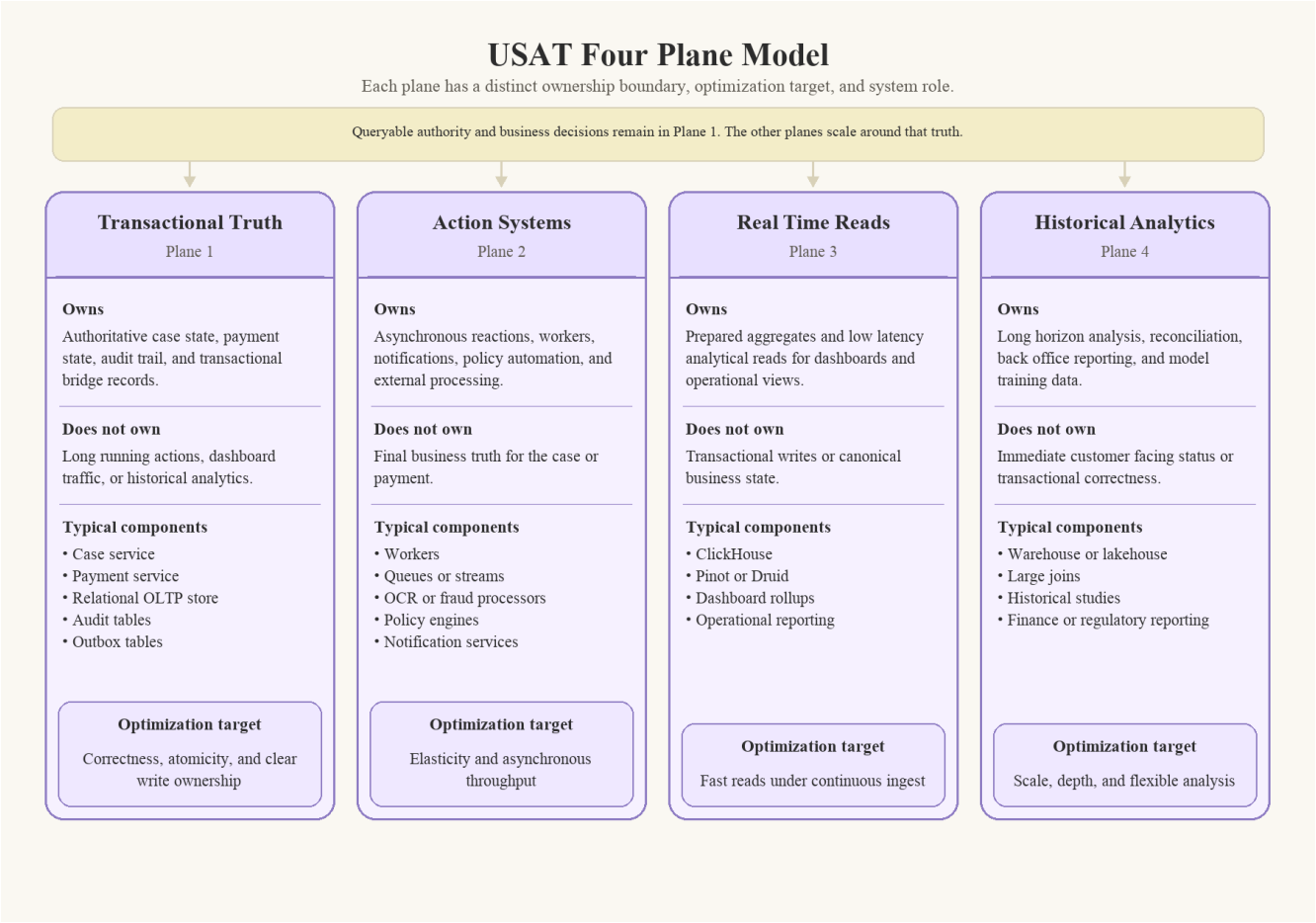

Figure 3: USAT four plane model and ownership boundaries

- Plane 4 owns long horizon analysis and historical depth.

Without this separation, teams tend to collapse transactional concerns, action concerns, and analytical concerns into one architecture, or they spread core business decisions across infrastructure layers that were never meant to own them. Both are failure modes USAT tries to prevent.

11.1 Plane 1 Transactional Core

The transactional core owns authoritative business truth.

- Typical technology: PostgreSQL or another relational OLTP store
- Responsibilities: state transitions, invariants, audit records, outbox writes
- Optimization target: correctness, low latency local commits, clear write ownership

This plane should remain narrow. It is the place for business truth, not for arbitrary side effects or heavy analytics.

In AFRP, Plane 1 would contain the canonical document case state, risk decision acceptance, payment authorization state, settlement acceptance, and the audit and outbox records associated with those changes.

11.2 Plane 2 Event Driven Actions

This plane consumes committed events and performs asynchronous reactions.

- Typical technology: Kafka, queue systems, stream processors, background workers
- Responsibilities: fraud scoring, alerting, policy reactions, notifications, downstream automation
- Optimization target: elasticity, asynchronous processing, autonomous reactions

Stream processors such as Flink are useful here when the job is to detect patterns and trigger automated actions. The key point is that this plane should react to committed truth rather than define the truth of the core transaction itself (see Reference 9).

In AFRP, Plane 2 would include OCR processing, fraud scoring, alert generation, duplicate document detection, policy reactions, and notifications to internal reviewers or downstream systems.

11.3 Plane 3 Real Time Analytical Read Path

This plane serves low latency dashboards and high concurrency analytical reads.

- Typical technology: ClickHouse, Pinot, Druid, or PostgreSQL with a real time analytical extension when the workload remains narrower
- Responsibilities: aggregated metrics prepared in advance, dashboard queries, customer and executive reporting
- Optimization target: fast reads under sustained ingest

This plane should be optimized for reading, not for being the source of business truth. Modern PostgreSQL stacks can go much further here than older architecture advice often assumes, especially when they add continuous aggregates, columnar storage, partitioning, or other real time analytical extensions. For teams with moderate scale and a relatively narrow dashboard workload, that can be a very good Plane 3 choice because it preserves operational simplicity.

The distinction still matters, however. Once the read path becomes heavily analytical in concurrency, retention, scan volume, or aggregation breadth, a purpose built analytical engine remains the clearer fit. ClickHouse materialized view style aggregation is one representative example of that design point (see Reference 10).

In AFRP, Plane 3 would serve dashboards such as approval funnel counts, payment volume summaries, settlement lag, flagged case throughput, reviewer workload, and customer visible payment status timelines.

11.4 Plane 4 Batch Analytics and Historical Depth

This plane serves long range analysis, back office reporting, model training, and deep historical queries.

- Typical technology: BigQuery, Snowflake, Redshift, data lakehouse systems
- Responsibilities: ad hoc analysis, large joins, historical depth, data science workloads
- Optimization target: scale and flexibility over per request latency

This plane is often a poor fit for customer facing dashboard traffic, but it is the right place for large historical analysis.

In AFRP, Plane 4 would support monthly reconciliation, fraud model training datasets, longitudinal customer risk studies, retention analysis, and regulatory or finance reporting over long historical ranges.

12. Read Path Discipline

USAT separates write concerns from read concerns deliberately.

The transactional database should not become the general purpose engine for multi year analytics or high concurrency dashboards. That does not mean every analytical query will "crash" the OLTP store. It means mixed workloads typically create avoidable latency variance, resource contention, and operational risk.

For that reason:

- committed events and selected CDC streams feed the analytical systems,
- real time dashboards read from Plane 3,
- long range analysis reads from Plane 4,
- the core OLTP system remains focused on correct writes and narrow operational reads.

13. Implementation Rules

USAT can be summarized by a small number of implementation rules:

1. Identify the business invariants before choosing the deployment boundary.
2. Keep core business rules in cohesive services rather than distributing them across transport and orchestration systems.
3. Keep state transitions that must remain mutually consistent inside one local transaction whenever practical.
4. Make authoritative business truth directly queryable and auditable from the transactional system of record.
5. Write outbound intent in the same transaction as the state mutation.
6. Treat external publication as asynchronous delivery of already committed intent.
7. Separate public event contracts from internal table schemas.
8. Accept external signals only by converting them into one new authoritative local commit.
9. Keep analytics and asynchronous actions outside the transactional write path.
10. Preserve operational legibility so current state and pending actions remain explainable without reconstructing the business process from infrastructure history.

14. Tradeoffs and Limits

USAT is not a universal prescription.

It is strongest when:

- the business contains high value invariants,
- correctness matters more than maximal deployment independence,
- the number of strongly coupled transactional operations is small enough to keep the core narrow,
- the system still needs large downstream analytical or asynchronous capabilities.

It is less compelling when:

- the domains are naturally independent,
- eventual consistency is acceptable in the user visible workflow,
- cross domain atomicity is not a real business requirement,
- the organizational cost of a shared core outweighs the consistency benefit.

The model therefore recommends deliberate centralization only where invariants justify it.

15. Comparison Criteria for Common Alternatives

USAT is easiest to evaluate when compared against a few common alternatives on explicit criteria rather than broad ideology.

Approach	Main strength	Main weakness	Best fit
USAT / Intent First Transactional Core	Strong business truth, clear state ownership, better troubleshooting, and deliberate scaling around the core	Requires discipline about what belongs in the core and what must remain asynchronous	Systems with high value invariants and meaningful operational complexity
Service per Database plus Saga Coordination	High deployment autonomy and clear service ownership	More inconsistency windows, more compensation logic, and harder end to end reasoning	Naturally independent domains where eventual consistency is acceptable
Queue First / Fully Async Workflow	Strong decoupling and elastic ingestion	Weaker synchronous UX and more difficulty exposing immediate authoritative state	Fire and forget or ingestion heavy workflows where immediate entity truth is not required
Single OLTP for Writes and Reads	Simple initial architecture and fewer moving parts	Read workloads can distort write path latency and operational safety over time	Small systems with modest scale and limited analytical pressure

The comparison is not intended to declare one universal winner. It is intended to make the decision rule explicit: when correctness, legibility, and authoritative state matter more than maximum deployment autonomy, USAT becomes more attractive.

16. Conclusion

The main argument of USAT is straightforward: software architecture should be organized around business intent, not around the accidental shape of infrastructure.

Queues, workflow engines, object storage, stream processors, and analytical warehouses are all valuable. The mistake is allowing those tools to become the place where the business process actually lives.

A system remains easier to reason about when it has:

- one authoritative and queryable center for business truth,
- cohesive services that own the important business decisions,
- one explicit bridge from committed truth to asynchronous distribution,
- and clearly separated planes for actions, real time reads, and historical analytics.

USAT is therefore not against cloud systems and not against distribution. It is a proposal for where distribution should begin: after the system has already established, in one durable place, what it believes to be true.

17. References

1. Martin Fowler, "Monolith First." https://martinfowler.com/bliki/MonolithFirst.html
2. Martin Fowler, "CQRS." https://martinfowler.com/bliki/CQRS.html
3. Martin Fowler, "What do you mean by 'Event-Driven'?" https://martinfowler.com/articles/201701-event-driven.html
4. Chris Richardson, "Pattern: Transactional outbox." https://microservices.io/patterns/data/transactional-outbox
5. Chris Richardson, "Pattern: Saga." https://microservices.io/patterns/data/saga.html
6. AWS Prescriptive Guidance, "Transactional outbox pattern." https://docs.aws.amazon.com/prescriptive-guidance/latest/cloud-design-patterns/transactional-outbox.html
7. Debezium Documentation, "Outbox Quarkus Extension." https://debezium.io/documentation/reference/stable/integrations/outbox.html
8. Debezium Documentation, "Outbox Event Router." https://debezium.io/documentation/reference/stable/transformations/outbox-event-router.html

9. Apache Flink Documentation, "Overview." https://nightlies.apache.org/flink/flink-docs-stable/docs/concepts/overview/
10. ClickHouse Documentation, "Materialized Views." https://clickhouse.com/docs/materialized-views

Appendix A Key Terms

- **Business Invariant:** A statement that must remain true for the business to remain correct, regardless of infrastructure shape.
- **Authoritative State:** The state the system recognizes as the source of truth for current business meaning.
- **Service Owned Business Logic:** The principle that business rules and state transitions should live in cohesive application services rather than in transport or orchestration systems.
- **Transactional Center:** The narrow part of the architecture where mutually dependent state transitions are committed together.
- **Transactional Core:** The narrow part of the system that owns authoritative business state and enforces invariants.
- **Transactional Outbox:** A table written in the same local transaction as a business mutation, used to persist outbound publish intent durably.
- **Implanted Architecture:** A failure mode in which core business logic is expressed mainly through distributed infrastructure choreography instead of a cohesive application state model.
- **Operational Legibility:** The ability for engineers and operators to explain current business state and pending actions from the authoritative system of record.
- **Public Event Contract:** The stable event shape intentionally exposed to downstream consumers.
- **Domain CDC:** Internal change capture from domain tables, useful for replication or analytics but not automatically suitable as the public contract.
- **Four Plane Model:** The separation of transactional core, event driven actions, real time analytical reads, and batch analytics into different operational planes.